Inside the MySQL Clone Plugin

how it was built, and how it compares to Galera SST

Matthias Crauwels
Enterprise Customer Engineer, PlanetScale
@mcrauwel

Who am I?

- Living in Ghent, Belgium
- ~30 years Linux user / admin
- ~15 years MySQL DBA
- Enterprise Customer Engineer at PlanetScale
- Day job: Vitess, MySQL and Postgres at scale for large customers
- Clone shows up in my world as provisioning, not as a paper

Why this talk

- Everyone knows the **syntax**: `CLONE INSTANCE FROM ...`
- Far fewer people know the **protocol** underneath
- The interesting engineering is not the SQL, it is:
  - how you copy a living InnoDB without freezing it
  - what "consistent" means when files are copied over minutes
- And Galera solved a neighbouring problem very differently

Agenda

- Why clone exists
- Architecture: plugin and InnoDB
- The three-stage protocol
- Page tracking & redo archiving
- Remote clone & replication coordinates
- Galera SST / IST
- Side by side
- Tradeoffs

Why clone exists

The problem

Provision a **physical, crash-consistent** InnoDB data directory from a **live** server

- Concurrent DML must keep running
- Donor must not be blocked for long
- Result must boot as an ordinary mysqld
- One mechanism for local, remote and cluster-join provisioning

Design goals from the worklog

- Do not block the donor for long
  - target was **under a second** of hard blocking
- Donor performance impact no worse than **MEB**
- Work with concurrent DML
- Worklog-era limits: no DDL during clone, no encrypted tablespaces
  - encryption landed before GA (WL#9682); concurrent DDL only in 8.0.27

What already existed

- `mysqldump` — logical, slow, consistency is awkward
- Filesystem / volume snapshots — need the storage stack, still need a redo story
- XtraBackup / MEB — external binary, backup locks, not callable in-server
- Stop a replica and copy files — downtime and operational friction

Clone's bet

Put the physical snapshot **inside InnoDB**,
expose it through a storage engine API,
and let a plugin move the bytes

The worklogs

- **WL#9209** — local clone: stages, page tracking, redo archiving
- **WL#9210** — remote clone
- **WL#9211** — replication coordinates
- **WL#11636** — remote provisioning, in-place
- Shipped in **MySQL 8.0.17** as `mysql_clone.so`

Architecture

Two halves

- **Clone plugin** (`mysql_clone.so`)
  - SQL command, session, worker tasks
  - network protocol, resume after failure
  - the actual byte copy (read/write; `sendfile` for local clone only)
- **InnoDB**
  - snapshot state machine
  - page tracking, redo archiving
  - apply into the destination data directory

The layering

```
SQL         CLONE LOCAL ... / CLONE INSTANCE FROM ...
                          |
plugin      session, tasks, network, resume, byte copy
                          |
handlerton  begin / copy / ack / end
            apply_begin / apply / apply_end
                          |
InnoDB      snapshot -> clone handle -> task(s)
            page tracking + redo archiving
```

The plugin moves bytes. InnoDB defines "consistent".

The handlerton API, both sides

```
// donor
clone_begin(locator, type, mode)       // start / resume / attach
clone_copy(locator, callbacks)         // produce snapshot data
clone_ack(locator, ...)                // recipient progress & errors
clone_end(locator, ...)
// recipient
clone_apply_begin(locator, data_dir, mode)
clone_apply(locator, callbacks)        // apply next chunk
clone_apply_end(locator, ...)
```

- **Locator** — non-persistent ID of this snapshot; carries apply progress, which is how resume works
- **Mode** — start, restart, add task
- `clone_copy` and `clone_apply` are producer and consumer loops, not socket I/O
- Local clone: one process bridges copy to apply. Remote: two servers, plugin protocol between
- Failure is all-or-nothing: finish and recover, or discard and roll back

Two callbacks, and why

- `clone_file_cbk(fd, length)`
  - bulk tablespace copy straight from the file on disk
- `clone_buffer_cbk(buffer, length)`
  - pages already in memory, redo chunks
- Every chunk carries a **descriptor**
  - which space, which page, which stage
  - the plugin ships bytes, it does not interpret InnoDB

The three stages

The state machine

```
INIT  ->  FILE COPY  ->  PAGE COPY  ->  REDO COPY  ->  DONE
              |              |              |
     all tablespace    pages changed   archived redo
     files, tracking   during file     after tracking
     turned on         copy, sorted    stopped
```

- Bulk copy for throughput
- Page delta to repair what moved underneath it
- Redo tail to pin **one** LSN

// The three stages

Stage 1 — file copy

- Start **page tracking** at `CLONE_START_LSN`
  - the LSN of the last finished mini-transaction
- Copy every InnoDB tablespace file as it is on disk
- DML keeps running, so those files end up **torn**
- Record every page flushed during this window

Stage 1 — the invariant

- Pages already flushed before tracking started
  - captured by the **file copy** bytes
- Pages dirtied and flushed **during** file copy
  - captured by **page tracking**, sent in stage 2
- Pages still dirty in memory
  - captured by **redo** in stage 3

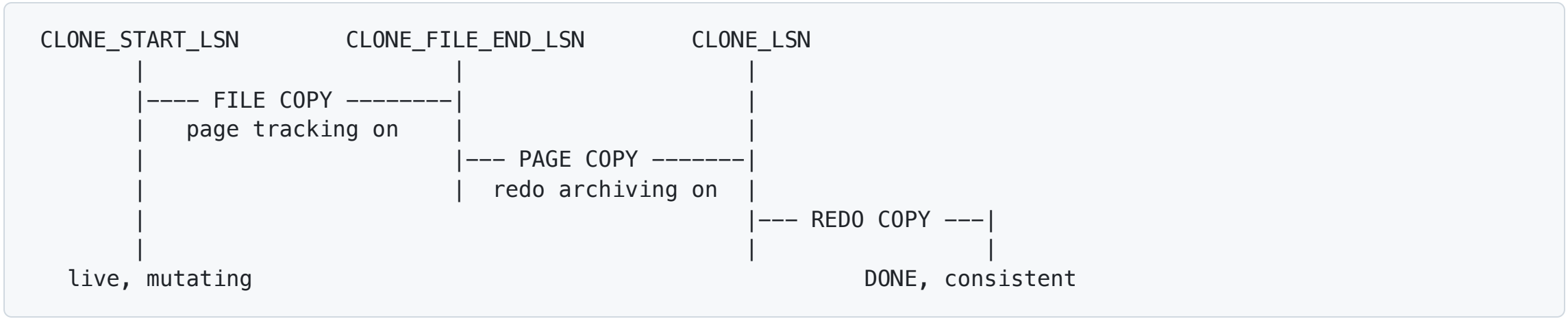
Stage 2 — page copy

- Force a **checkpoint**, start **redo archiving** from that checkpoint LSN
- Then stop page tracking at `CLONE_FILE_END_LSN`
- Re-send every tracked page
  - preferably straight from the **buffer pool**
  - sorted by space id and page id
- Recipient overwrites those pages in the copied files

Stage 3 — redo copy

- Stop redo archiving at `CLONE_LSN`
  - **this is the LSN of the cloned database**
- Ship archived redo for `[handoff checkpoint LSN .. CLONE_LSN]`
- On first start, ordinary **InnoDB crash recovery** applies it
- As consistent as a crash at `CLONE_LSN`

The timeline



- File copy is a **smear**, not a point in time
- Page copy is a **set of pages**, not an LSN
- Only after redo copy do you have **one** LSN

// The three stages

Why not just lock and copy?

- Long read lock plus copy
  - simple, easy to prove, and unacceptable on a primary
- External hot backup tool
  - process boundary, backup locks, no in-server API
- Clone's hybrid
  - short critical sections, engine-native tracking, DML continues

Page tracking & redo archiving

Tracking on flush, not on dirty

- Two candidate hook points
  - when a mini-transaction dirties a page
  - when the page is **written to disk**
- They chose flush time
  - keeps the mini-transaction path out of it
  - lines up with checkpoint semantics
- Cost: you see the flush, not every modification

Tracking mechanics

- **Producer** — the flush hook records `(space_id, page_id)` on every page write
  - frame LSN check skips a page already flushed this window
- **Buffer** — fixed in-memory ring, 32 × 16 KiB blocks
- **Consumer** — page archiver thread spills blocks to files, headers carry start and end LSN
- **Customer** — client API: start, stop, fetch pages, release (deletes the files)
- Ring full? the flusher waits, up to 30 min, then the **clone** fails, not the donor
- Later made persistent and reused by **MEB** — still **not** incremental clone

Redo archiving

- **Why** — the redo log is a ring; page copy can take long enough for it to wrap
- **Producer** — the normal log writer, unchanged
- **Consumer** — log archiver thread copies redo to side files from the handoff checkpoint
  - **archived LSN** = how far redo has been safely copied out
- **Customer** — same API: start, stop (fixes `CLONE_LSN`), fetch, release
- Writer about to wrap onto unarchived redo? waits at most ~1 s, then the **clone** fails
- Recipient lays the redo down as its own log; first start = ordinary crash recovery

Remote clone & coordinates

Three modes

- **Local** — `CLONE LOCAL DATA DIRECTORY = …`: same server, new directory, no network
- **Remote to a directory** — `CLONE INSTANCE FROM … DATA DIRECTORY = …`: recipient keeps running, start a second mysqld on the result
- **Remote in-place** — `CLONE INSTANCE FROM …` with no directory: recipient swaps its data directory and restarts (needs a supervisor)
- All three are initiated from the recipient

Resume, not incremental

- Network failure does not restart the whole clone
- Locator carries acknowledged apply progress
- Restart mode reattaches to the **same snapshot**
- If the snapshot is gone, you start over

Replication coordinates

- A physical copy alone is **not** a replica
- Binlog position
  - ordered commit forced, unordered commits drained
  - XA blocked at the end of redo copy so binlog matches redo
- GTID taken consistent with the last committed transaction
- Exposed through `performance_schema.clone_status`

Who calls clone

- A DBA provisioning a replica by hand
- **Group Replication / InnoDB Cluster** joining a member
- Later: **Galera clusters** (Codership 2020, Percona 2025), with clone as an SST method
- Same engine protocol, different orchestrators

Galera SST / IST

What Galera is solving

- Make this node equal to cluster state `UUID:seqno`
- Not "give me InnoDB at LSN X"
- Two ways to get there
  - **IST** — stream the missing write-sets
  - **SST** — full state snapshot, then catch up

IST and the GCache

- GCache is a ring buffer of recent write-sets on every node
- Joiner's seqno still in the donor's GCache
  - **IST**: stream the delta, apply, synced
- Otherwise
  - **SST**: full snapshot, then apply what happened during it
- `gcache.size` is often the difference between **seconds and hours**

SST methods

- **rsync** — FTWRL held on the donor for the entire copy
- **xtrabackup-v2 / mariabackup** — hot backup plus redo, mostly online
- **mysqldump** — logical, slow, blocking
- **clone** — the MySQL clone plugin as the SST backend (MySQL-wsrep 8.0.22, default in 8.4; PXC 8.0.41)
- Galera **orchestrates**; the method **implements the bytes**

Side by side

Clone vs Galera SST

	Clone	Galera SST
Purpose	provision an instance or cluster member	bring a node to cluster state `UUID:seqno`
Identity	InnoDB LSN	write-set sequence number
Driver	SQL and a plugin, inside the server	wsrep provider plus external scripts
Incremental	none in the plugin; GR streams binlog first	**IST** from the GCache when possible
Transport	built into the plugin	scripted, typically a pipe tool
Coupling	deep InnoDB integration	the method must understand the data directory

// Side by side

One sentence each

Clone — give me InnoDB at LSN X, crash-recoverable

Galera — make this node equal to cluster state `UUID:seqno` Y

// Side by side

When the two meet

- Galera decides: can this joiner do **IST**?
- If not, it invokes the configured SST method
- With `wsrep_sst_method=clone`, that method is the **clone plugin**
  - file copy, page copy, redo copy
  - joiner boots at `CLONE_LSN`, recovers its wsrep position from InnoDB
- Then Galera applies the write-sets it missed

Tradeoffs

Choices worth arguing about

- Tracking at flush time rather than at dirty time
- Three stages instead of a backup lock and a copy
- A storage engine API used by essentially one engine
- Page tracking built as a platform, later reused by MEB
- No incremental path for a fresh data directory in async MySQL
- Bounded redo stall, then drop the clone

Takeaways

- Clone is a **consistency protocol**, not a file copy
- The real engineering is **page tracking and redo archiving**
- Galera's cleverness is **IST**; SST is the expensive fallback
- Compare the **problems**, not the transfer speeds
- The modern shape: cluster orchestration plus an engine-native snapshot

Where to read the code

```
plugin/clone/              transport, SQL, network, resume
storage/innobase/clone/
  clone0snapshot.cc        the stage machine
  clone0copy.cc            donor side
  clone0apply.cc           recipient side
  clone0desc.cc            chunk descriptors
storage/innobase/arch/
  arch0page.cc             page tracking
  arch0log.cc              redo archiving
```

References

- WL#9209 — InnoDB: Clone local replica; WL#9210, WL#9211, WL#11636
- WL#9682 — InnoDB: Support cloning encrypted and compressed database
- D. Banerjee, "Clone: Create MySQL instance replica" (2019)
- D. M N, "InnoDB Clone and page tracking" (2020)
- facebook/mysql-5.6 wiki, "Clone Plugin Background and Internals" (as of 8.0.28)
- Pep Pla, "The MySQL Clone Wars: Plugin vs. Percona XtraBackup" (2021)
- K. Bauskar, "Understanding how an IST donor is selected" (2017)
- Codership, PXC and MariaDB docs — SST methods, IST, GCache

Questions?

file copy, page copy, redo copy — one LSN

IST if you can, SST if you must

Thank you!

Matthias Crauwels

Enterprise Customer Engineer, PlanetScale

@mcrauwel